

**COMPUTER VISION & IMAGE PROCESSING**  
**HISTOGRAM**



**KHEN MUHAMMAD CAHYO**

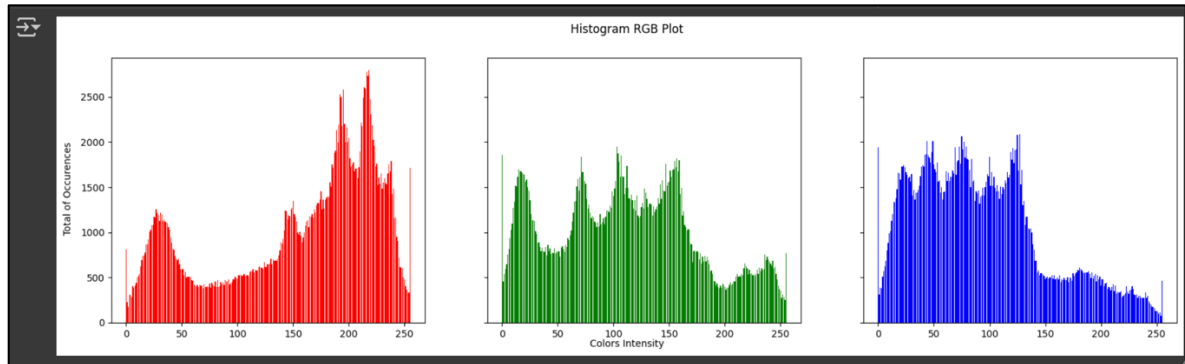
**244107023002**

**TI-3I**

**STATE POLYTECHNIC OF MALANG**  
**INFORMATION TECHNOLOGY DEPARTMENT**

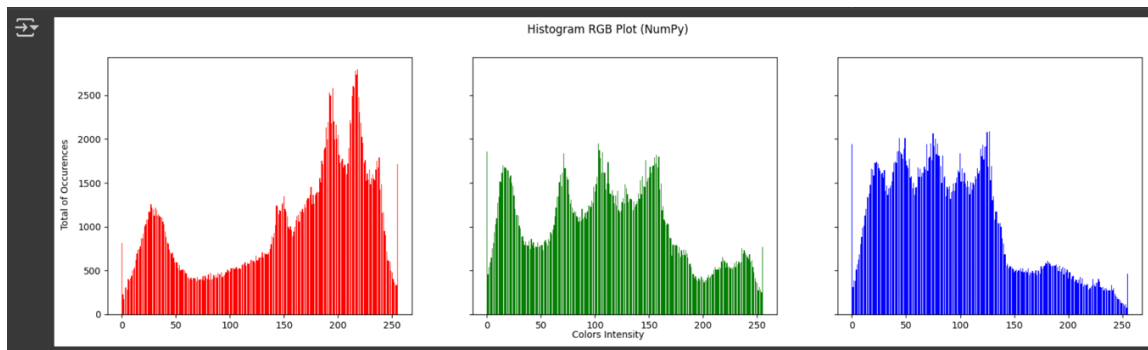
## D1 PRACTICUM – HISTOGRAM

- **Create Manual Histogram**



- **Number 1 Task – NumPy Histogram**

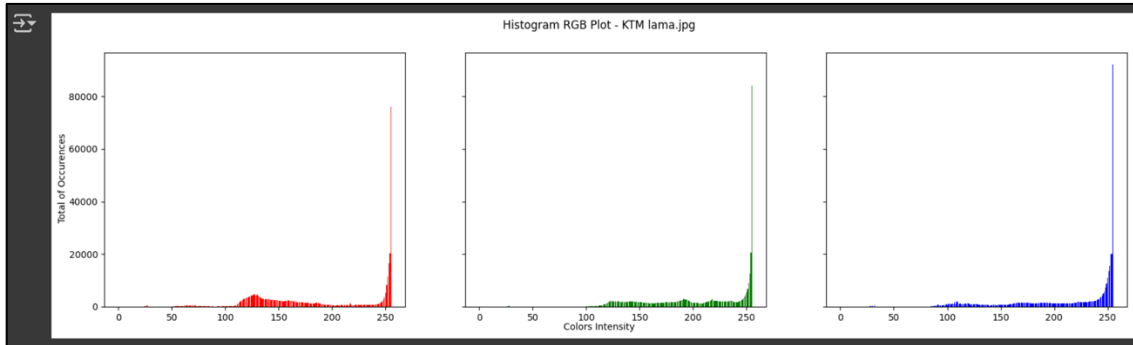
```
1 img = cv.imread('lena.jpg')
2 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
3
4 # Separated channel colors
5 r, g, b = img[:, :, 0], img[:, :, 1], img[:, :, 2]
6
7 # Count histogram using numpy
8 hist_r, bins_r = np.histogram(r, bins=256, range=(0, 256))
9 hist_g, bins_g = np.histogram(g, bins=256, range=(0, 256))
10 hist_b, bins_b = np.histogram(b, bins=256, range=(0, 256))
11
12 # Plotting
13 names = np.arange(256)
14 fig, axs = plt.subplots(1, 3, figsize=(20, 5), sharex=True, sharey=True)
15 fig.suptitle("Histogram RGB Plot (NumPy)")
16
17 fig.text(0.09, 0.5, 'Total of Occurrences', va='center', rotation='vertical')
18 fig.text(0.5, 0.04, 'Colors Intensity', ha='center')
19
20 axs[0].bar(names, hist_r, color='red')
21 axs[1].bar(names, hist_g, color='green')
22 axs[2].bar(names, hist_b, color='blue')
23
24 plt.show()
```



Histograms calculated using manual loops and `numpy.histogram` produce the same output, as they both calculate the frequency of pixel intensity occurrences within the range 0–255 for each color channel. The only difference lies in the calculation method: manual loops require nested loops for each pixel, while `numpy.histogram` is more efficient due to library optimization.

- Task Number 2 – KTM Lama

```
1 # Read image
2 img = cv.imread('KTM lama.jpg')
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 # Separated channel colors
6 r, g, b = img[:, :, 0], img[:, :, 1], img[:, :, 2]
7
8 # Count histogram results using numpy
9 hist_r, bins_r = np.histogram(r, bins=256, range=(0, 256))
10 hist_g, bins_g = np.histogram(g, bins=256, range=(0, 256))
11 hist_b, bins_b = np.histogram(b, bins=256, range=(0, 256))
12
13 # plotting
14 names = np.arange(256)
15 fig, axs = plt.subplots(1, 3, figsize=(20, 5), sharex=True, sharey=True)
16 fig.suptitle("Histogram RGB Plot - KTM lama.jpg")
17
18 fig.text(0.09, 0.5, 'Total of Occurrences', va='center', rotation='vertical')
19 fig.text(0.5, 0.04, 'Colors Intensity', ha='center')
20
21 axs[0].bar(names, hist_r, color='red')
22 axs[1].bar(names, hist_g, color='green')
23 axs[2].bar(names, hist_b, color='blue')
24
25 plt.show()
```

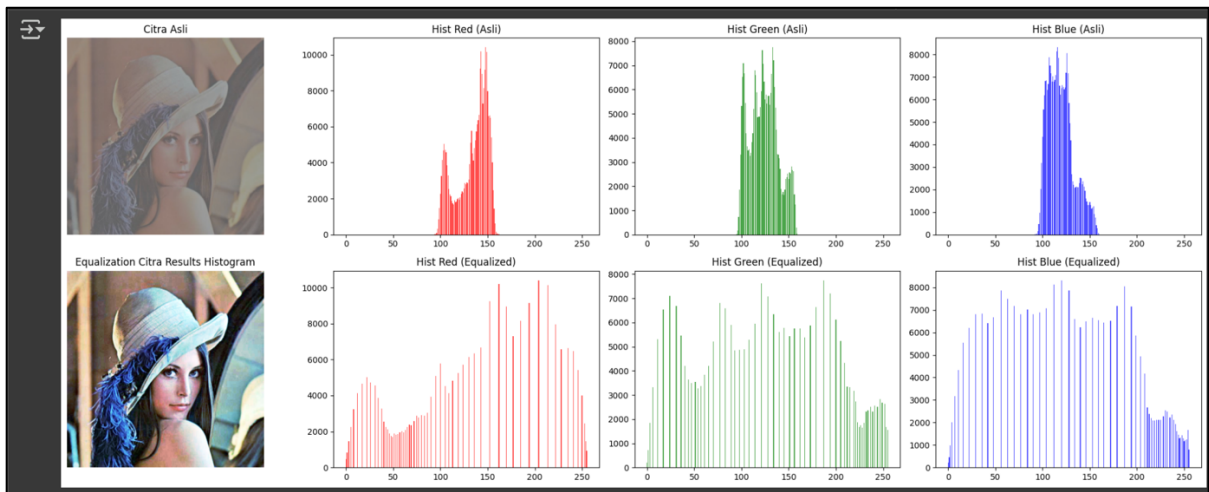


From the histogram of the image KTM lama.jpg it can be seen that the majority of the intensity distribution in the three channels (red, green, and blue) accumulates in the high intensity area (close to 255), while in the low intensity section only a few values appear. This indicates that the image tends to be bright, because most of the pixels are in the very bright/white color range. In addition, the distribution of the histogram is not even across the range (more concentrated at the right end), so this image can also be said to have low contrast, because the variation in color intensity is not evenly distributed from dark to light.

## D2 PRACTICUM – EQUALIZATION HISTOGRAM

- Task Number 1 – Compare Equalization Histogram Results

```
1 # Read image
2 img = cv.imread('lena_lc.jpg')
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 # Separated channel colors
6 r, g, b = cv.split(img)
7
8 # Histogram Equalization for each channels
9 r_eq = cv.equalizeHist(r)
10 g_eq = cv.equalizeHist(g)
11 b_eq = cv.equalizeHist(b)
12
13 # Merge back equalization channel results
14 img_eq = cv.merge((r_eq, g_eq, b_eq))
15
16 # Histogram plotting function
17 def plot_hist(channel, color):
18     hist, bins = np.histogram(channel, bins=256, range=(0,256))
19     plt.bar(np.arange(256), hist, color=color, alpha=0.7)
20
21 # Plot results before and after
22 fig, axs = plt.subplots(2, 4, figsize=(20, 8))
23
24 # Real citra
25 axs[0,0].imshow(img)
26 axs[0,0].set_title("Citra Asli")
27 axs[0,0].axis("off")
28
29 # Real citra histogram
30 plt.sca(axs[0,1]); plot_hist(r, 'red'); axs[0,1].set_title("Hist Red (Asli)")
31 plt.sca(axs[0,2]); plot_hist(g, 'green'); axs[0,2].set_title("Hist Green (Asli)")
32 plt.sca(axs[0,3]); plot_hist(b, 'blue'); axs[0,3].set_title("Hist Blue (Asli)")
33
34 # Equalization citra results
35 axs[1,0].imshow(img_eq)
36 axs[1,0].set_title("Equalization Citra Results Histogram")
37 axs[1,0].axis("off")
38
39 # Equalization citra results histogram
40 plt.sca(axs[1,1]); plot_hist(r_eq, 'red'); axs[1,1].set_title("Hist Red (Equalized)")
41 plt.sca(axs[1,2]); plot_hist(g_eq, 'green'); axs[1,2].set_title("Hist Green (Equalized)")
42 plt.sca(axs[1,3]); plot_hist(b_eq, 'blue'); axs[1,3].set_title("Hist Blue (Equalized)")
43
44 plt.tight_layout()
45 plt.show()
```

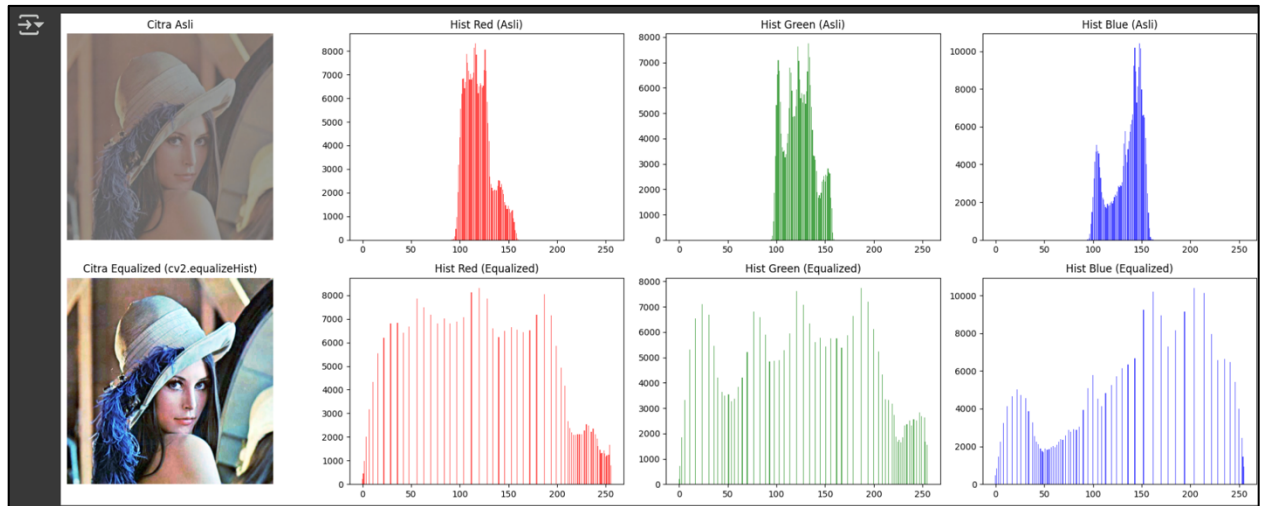


- **Task Number 2 – Equalization Histogram CV2**

```

8 # Equalization Histogram for each channel using cv2.equalizeHist
9 b_equalized = cv.equalizeHist(b)
0 g_equalized = cv.equalizeHist(g)
1 r_equalized = cv.equalizeHist(r)

```



The results of manual histogram equalization (following the previous flowchart) and using `cv2.equalizeHist()` are essentially the same, as both implement the concept of smoothing the pixel intensity distribution by utilizing the cumulative function of the histogram. The only difference is in the implementation method. Manual > we calculate the frequency, cumulative, normalization, and transformation ourselves. `Cv2.equalizeHist` > OpenCV already provides a more concise and efficient built-in function.

## D2 PRACTICUM QUESTIONS

### Number 1 Answers

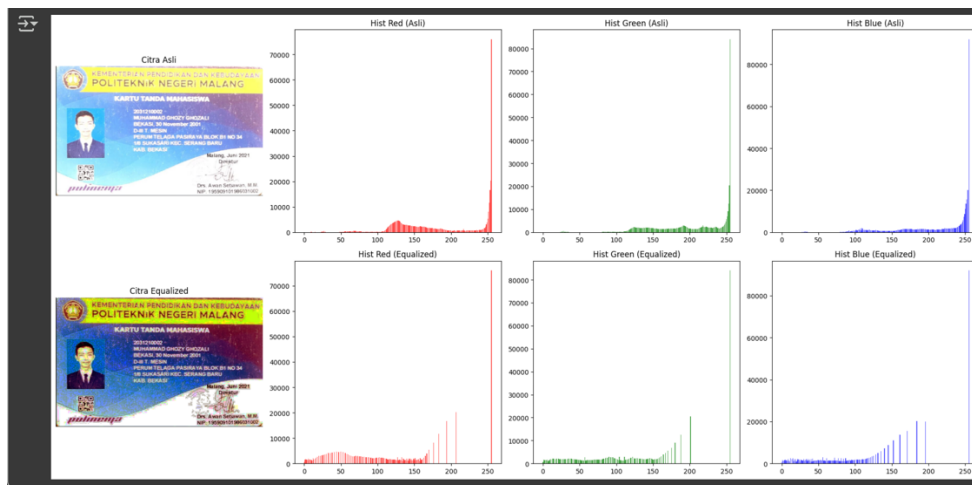
```
1 # Read image
2 img = cv.imread('lena.jpg')
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 # Split channel colors
6 r, g, b = cv.split(img)
7
8 # Equalization on each channels
9 r_eq = cv.equalizeHist(r)
10 g_eq = cv.equalizeHist(g)
11 b_eq = cv.equalizeHist(b)
12
13 # Merge back equalization results
14 img_eq = cv.merge((r_eq, g_eq, b_eq))
15
16 # PSNR function
17 def psnr(img1, img2):
18     mse = np.mean((img1 - img2) ** 2)
19     if mse == 0:
20         return float('inf')
21     PIXEL_MAX = 255.0
22     return 20 * np.log10(PIXEL_MAX / np.sqrt(mse))
23
24 # Count PSNR between real citra dan equalization results
25 psnr_value = psnr(img, img_eq)
26 print("PSNR between real citra and equalized:", psnr_value)
27
28 # Plot hasil citra
29 fig, axs = plt.subplots(1, 2, figsize=(12, 6))
30 axs[0].imshow(img)
31 axs[0].set_title("Real Citra")
32 axs[0].axis("off")
33
34 axs[1].imshow(img_eq)
35 axs[1].set_title("Equalized Citra")
36 axs[1].axis("off")
37 plt.show()
```



The calculation results show that the PSNR value = 28.46 dB, which is considered moderate (not too high, but not too low). This means that there is a significant difference between the original image and the equalized image, but the visual quality is still well maintained. The main change is seen in the distribution of pixel intensity which has become more even, so that details in dark and light areas are clearer than in the original image. In other words, although mathematically there is a decrease in similarity (PSNR is not very high), visually the histogram equalization actually improves the readability of image details and improves contrast.

## Number 2 Answers

```
1 # Read image
2 img = cv.imread('KTM lama.jpg')
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 # Split channels
6 r, g, b = cv.split(img)
7
8 # Histogram equalization per channel
9 r_eq = cv.equalizeHist(r)
10 g_eq = cv.equalizeHist(g)
11 b_eq = cv.equalizeHist(b)
12
13 # Merge equalized channels
14 img_eq = cv.merge([r_eq, g_eq, b_eq])
15
16 # Fungsi plotting histogram
17 def plot_hist(channel, color):
18     hist, bins = np.histogram(channel, bins=256, range=(0, 256))
19     plt.bar(np.arange(256), hist, color=color, alpha=0.7)
20
21 # Plot results
22 fig, axs = plt.subplots(2, 4, figsize=(20, 10))
23
24 # --- Row 1: Real Citra & histogram ---
25 axs[0,0].imshow(img)
26 axs[0,0].set_title("Citra Asli")
27 axs[0,0].axis("off")
28
29 plt.sca(axs[0,1]); plot_hist(r, 'red'); axs[0,1].set_title("Hist Red (Asli)")
30 plt.sca(axs[0,2]); plot_hist(g, 'green'); axs[0,2].set_title("Hist Green (Asli)")
31 plt.sca(axs[0,3]); plot_hist(b, 'blue'); axs[0,3].set_title("Hist Blue (Asli)")
32
33 # --- Row 2: Citra equalized & histogram ---
34 axs[1,0].imshow(img_eq)
35 axs[1,0].set_title("Citra Equalized")
36 axs[1,0].axis("off")
37
38 plt.sca(axs[1,1]); plot_hist(r_eq, 'red'); axs[1,1].set_title("Hist Red (Equalized)")
39 plt.sca(axs[1,2]); plot_hist(g_eq, 'green'); axs[1,2].set_title("Hist Green (Equalized)")
40 plt.sca(axs[1,3]); plot_hist(b_eq, 'blue'); axs[1,3].set_title("Hist Blue (Equalized)")
41
42 plt.tight_layout()
43 plt.show()
```



From the comparison of the KTM lama.jpg image before and after histogram equalization, it can be seen that the original image tends to be too bright with a dominance of intensity in high areas (close to white), so that some facial details, text, and background appear less clear. After equalization, the intensity distribution is more even so that details in dark and light areas become more visible, for example facial contours, small writing, and background patterns are more prominent. However, the side effect is that the image looks more “hard” or excessively contrasty, so that the colors look unnatural. In terms of information, equalization does improve the readability of details, but from a visual aesthetic point of view, the results can feel somewhat rough.



## D3 PRATICUM TASK – DITHERING

### Answer Number 1

```
1 # Read image as RGB
2 img = cv.imread("lena.jpg")
3 img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
4
5 # Convert to float for calculation
6 img = img.astype(float)
7
8 # Citra size
9 height, width, channels = img.shape
10
11 # Floyd-Steinberg dithering for each channels
12 for y in range(height - 1):
13     for x in range(1, width - 1):
14         for c in range(channels):
15             old_pixel = img[y, x, c]
16             new_pixel = 255 if old_pixel >= 128 else 0
17             img[y, x, c] = new_pixel
18             error = old_pixel - new_pixel
19
20             # Spread the errors to neighbors
21             img[y, x+1, c] += error * 7/16
22             img[y+1, x-1, c] += error * 3/16
23             img[y+1, x, c] += error * 5/16
24             img[y+1, x+1, c] += error * 1/16
25
26 # Convert back to uint8
27 img_dithered = np.clip(img, 0, 255).astype(np.uint8)
28
29 # Results plot
30 fig, axs = plt.subplots(1, 2, figsize=(12, 6))
31 axs[0].imshow(cv.cvtColor(cv.imread("lena.jpg"), cv.COLOR_BGR2RGB))
32 axs[0].set_title("Real Citra (RGB)")
33 axs[0].axis("off")
34
35 axs[1].imshow(img_dithered)
36 axs[1].set_title("Citra after Floyd-Steinberg Dithering (RGB)")
37 axs[1].axis("off")
38
39 plt.show()
```



Real Citra (RGB)



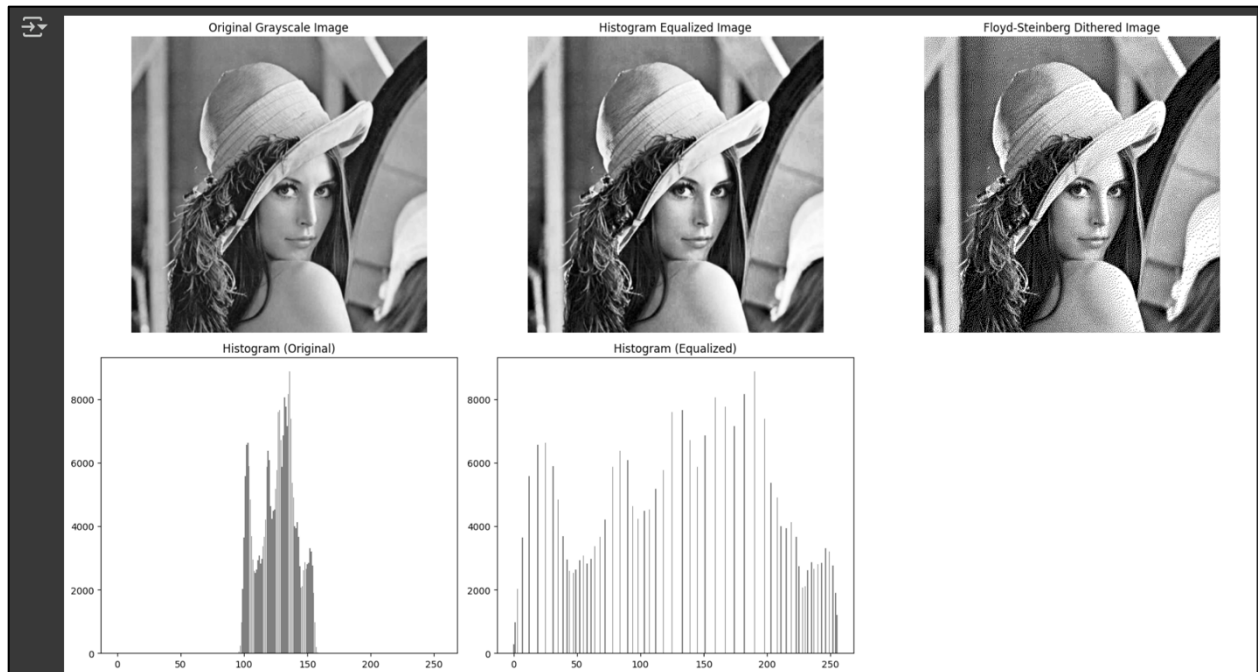
Citra after Floyd-Steinberg Dithering (RGB)





## Answer Number 2

```
1 # 1. Read image using cv.imread with grayscale
2 img = cv.imread("lena_lc.jpg", cv.IMREAD_GRAYSCALE)
3
4 # 2. Apply Histogram Equalization
5 img_eq = cv.equalizeHist(img)
6
7 # 3. Apply Floyd-Steinberg dithering on the equalized image
8 img_fs = img_eq.astype(float)
9 height, width = img_fs.shape
10
11 for y in range(height - 1):
12     for x in range(1, width - 1):
13         old_pixel = img_fs[y, x]
14         new_pixel = 255 if old_pixel >= 128 else 0
15         img_fs[y, x] = new_pixel
16         error = old_pixel - new_pixel
17
18         # Spread the error to the neighboring pixels
19         img_fs[y, x+1] += error * 7/16
20         img_fs[y+1, x-1] += error * 3/16
21         img_fs[y+1, x] += error * 5/16
22         img_fs[y+1, x+1] += error * 1/16
23
24 # Convert back to uint8
25 img_dithered = np.clip(img_fs, 0, 255).astype(np.uint8)
26
27 # 4. Plot the results
28 fig, axs = plt.subplots(2, 3, figsize=(18, 10))
29
30 # Original grayscale image
31 axs[0,0].imshow(img, cmap="gray")
32 axs[0,0].set_title("Original Grayscale Image")
33 axs[0,0].axis("off")
34
35 # Equalized image
36 axs[0,1].imshow(img_eq, cmap="gray")
37 axs[0,1].set_title("Histogram Equalized Image")
38 axs[0,1].axis("off")
39
40 # Dithered image
41 axs[0,2].imshow(img_dithered, cmap="gray")
42 axs[0,2].set_title("Floyd-Steinberg Dithered Image")
43 axs[0,2].axis("off")
44
45 # Histogram of original image
46 hist_orig, _ = np.histogram(img.ravel(), bins=256, range=(0,256))
47 axs[1,0].bar(np.arange(256), hist_orig, color="gray")
48 axs[1,0].set_title("Histogram (Original)")
49
50 # Histogram of equalized image
51 hist_eq, _ = np.histogram(img_eq.ravel(), bins=256, range=(0,256))
52 axs[1,1].bar(np.arange(256), hist_eq, color="gray")
53 axs[1,1].set_title("Histogram (Equalized)")
54
55 # Empty subplot for alignment
56 axs[1,2].axis("off")
57
58 plt.tight_layout()
59 plt.show()
```



**GitHub Link:**

<https://github.com/KhenCahyo13/MyCampus-Jobsheet/blob/smt-5/pcvk/week-5/Week5.ipynb>